



our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Colb Finance

Liquids

SECURITY REVIEW

Date: 7 December 2025

CONTENTS

1. About Shieldify	3
2. Disclaimer	3
3. About Colb Finance - Liquids	3
4. Risk classification	4
4.1 Impact	4
4.2 Likelihood	4
5. Security Review Summary	4
5.1 Protocol Summary	4
5.2 Scope	4
6. Findings Summary	5
7. Findings	6

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and have secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Vyper, Rust, Cairo, Move and Go.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About Colb Finance - Liquids

Colb is the first native non-custodial tokenization solution that enables peerless access to Swiss-grade wealth management strategies, pre-IPO opportunities, and premium investment funds. It offers a bankruptcy-remote Trust structure and native ownership of real-world assets, all on-chain. Colb reduces the entrance threshold to such investments by removing constraints such as the minimum investment amount to get exposure to them. The protocol is designed with security at its core, boasting compliance with Swiss regulations and DeFi composability. Colb envisions a future rooted in transparency where every individual has equitable access to premium RWA investments.

Overview

Colb Liquids is a system designed for the maintenance and management of diverse real-world assets (RWAs), enabling users to gain exposure to o-chain asset performance through the deposit of \$USC tokens. When users deposit \$USC, they receive liquid tokens that represent and track the value of specific RWAs. The performance of these assets is reflected through gradual token price appreciation, rather than explicit yield distribution. The system leverages custom oracles to provide accurate and transparent asset valuations, while managing deposits and redemptions through queue-based processing. Deposits enter a queue where operators process requests based on the latest oracle prices, converting \$USC into corresponding liquid token shares. Redemptions are available via a standard queue-based mechanism or through an instant redemption option that provides immediate liquidity at a higher fee.

Each liquid token represents fractional ownership of the underlying RWAs, with its price determined by real-time oracle feeds tracking RWA/USD exchange rates. As the underlying assets accrue value o-chain, this appreciation is reflected directly in the liquid token's price. Comprehensive access control, fee management, and liquidity governance mechanisms are built into the system to ensure orderly operations and robust risk management.

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol’s overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 11 days, with a total of 264 hours dedicated to the audit by three researchers from the Shieldify team.

Overall, the code is well-written. The audit report contributed by identifying three Medium and four Low severity issues. They’re mainly related to edge-case deposit flows, slippage exposure on queued operations, withdrawal constraints, configuration pitfalls and upgrade or storage layout oversights.

The Colb Finance team has done a great job with their test suite and provided support and responses to all of the questions that the Shieldify researchers had.

5.1 Protocol Summary

Project Name	Colb Finance - Liquids
Repository	contracts-v2
Type of Project	RWAs, Pre-IPO, Liquids
Security Review Timeline	11 days
Review Commit Hash	70f0f061b0c86e163d2f32933df74e9194bf5bc6
Fixes Review Commit Hash	83a525031252b101e86600420c51230e342f68aa

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
contracts/liquid/PreIPOStrategy.sol	35
contracts/liquid/RWAStrategy.sol	29
contracts/liquid/abstract/CoreStrategy.sol	324
contracts/liquid/abstract/AsyncStrategy.sol	111
contracts/liquid/controls/FeeManager.sol	97
contracts/liquid/controls/Controller.sol	169
contracts/liquid/LSFactory.sol	74
contracts/liquid/oracle/OracleFactory.sol	42
contracts/liquid/oracle/LSAggregatorV3.sol	89
contracts/liquid/oracle/LSPriceOracle.sol	36
Total	1006

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **Medium** issues: **3**
- **Low** issues: **4**
- **Info** issues: **6**

ID	Title	Severity	Status
[M-01]	User Will Need to Deposit <code>firstDepositAmount</code> Even After Depositing Once	Medium	Fixed
[M-02]	Queued Deposits/Withdrawals Exposed to Unbounded Fee & Price Slippage	Medium	Acknowledged
[M-03]	Users Cannot Withdraw Full Balance When It Falls Below <code>minWithdrawRetry</code>	Medium	Fixed
[L-01]	<code>FeeManager</code> : <code>maxDeposit</code> Can Be Misconfigured < <code>firstDepositAmount</code> (DoS on First Deposit)	Low	Fixed
[L-02]	<code>LSFactory</code> Emits Outdated Implementation Address After Beacon Upgrade	Low	Fixed
[L-03]	Missing Storage Gap in <code>AsyncStrategy</code>	Low	Fixed

[L-04]	Improper Naming of Withdraw Bounds (Deposit Bounds Are Checked in Assets)	Low	Fixed
[I-01]	Incomplete Events Across Strategy and Factory Modules	Info	Fixed
[I-02]	Deprecated OpenZeppelin Counters Library Usage	Info	Fixed
[I-03]	State Variable <code>lastUpdatedAt</code> Set But Never Exposed	Info	Fixed
[I-04]	Missing Shares Check in <code>CoreStrategy::deposit()</code>	Info	Fixed
[I-05]	<code>LSPriceOracle.sol\getQuote ()</code> Function Does Not Enforce Staleness	Info	Fixed
[I-06]	Misconfiguration Risk: <code>isConfigured()</code> / Account <code>isConfigured()</code> Must Be Manually Set in <code>FeeManager</code> Contract	Info	Acknowledged

7. Findings

[M-01] User Will Need to Deposit `firstDepositAmount` Even After Depositing Once

Severity

Medium Risk

Description

In the `CoreStrategy::deposit()` function, we check if a user has deposited for the first time. If he has not, we require him to deposit more or equal to `firstDepositAmount`:

```
if (!firstDeposits[sender] && assets < firstDepositAmount) {
    revert AmountLessThanMin(assets, firstDepositAmount);
}
```

However, if his first deposit hasn't been processed and the user wants to deposit for the second time, he will again need to deposit more or equal to `firstDepositAmount`, because this variable is set in `AsyncStrategy::processDeposits()`:

```
if (!firstDeposits[d.sender]) firstDeposits[d.sender] = true;
```

Location of Affected Code

File: [contracts/liquid/abstract/CoreStrategy.sol#L278-L280](#)

```
function deposit(uint256 assets, address receiver) public virtual {
    // code
    if (!firstDeposits[sender] && assets < firstDepositAmount) {
        revert AmountLessThanMin(assets, firstDepositAmount);
    }
}
```

Impact

If a user wants to deposit again, before his first deposit has been processed, he needs to pay once again an amount equal to or greater than his `firstDepositAmount`.

Recommendation

One recommendation is to put the `firstDeposits[sender] = true;` in the `CoreStrategy::deposit()` function. This can allow a user to deposit and then cancel instantly. This will allow him to kind of bypass the `firstDepositAmount`. Of course, he will lose tokens because of the cancellation fees.

If you are okay with the risk explained above, then this would be the recommendation. If not, we are open to a discussion.

Team Response

Fixed.

[M-02] Queued Deposits/Withdrawals Exposed to Unbounded Fee & Price Slippage

Severity

Medium Risk

Description

Deposits and withdrawals are queued in `AsyncStrategy`, but neither the fee rate nor the price is locked at the time the user commits funds.

Current flow:

- `deposit()` / `redeem()`:
 - Validate limits/access.
 - Queue the request in `AsyncStrategy` [`deposits` / `withdrawals`].
 - Do NOT store:
 - * the fee rate used,
 - * the price used,
 - * any user-provided slippage bounds.
- `processDeposits()` / `processWithdrawals()`:
 - Call `getFee(...)` which pulls current fee config from `FeeManager` (including per-account overrides).

- Convert between assets/shares using `_convertToShares()` / `_convertToAssets()` with the current price.
- Final shares/assets can differ heavily from what the user expected when they called `deposit()` / `redeem()`.

Because `FeeManager` config (global + per account) is fully mutable by privileged roles between those two moments, a user can:

- Deposit at time T0 when `depositFeeRate` is low, based on `previewDeposit()`.
- Get processed at time T1 after a fee bump or a targeted `AccountFeeConfig` override.
- End up with far fewer shares/assets than what they reasonably expected.

There is no on-chain way for the user to bind this (no `minShares`, `minAssets`, `maxFeeBps`, `deadline`, etc.).

Location of Affected Code

File: [contracts/liquid/abstract/CoreStrategy.sol#L258-L281](#)

```
/**
 * @inheritdoc ICoreStrategy
 */
function deposit(uint256 assets, address receiver) public virtual {
    Kernel memory _kernel = kernel;

    IController _controller = IController(_kernel.controller);

    _controller.checkDeposit(address(this));

    address sender = msg.sender;

    if (!_controller.hasAccess(sender, address(this)) || !_controller.
        hasAccess(receiver, address(this))) {
        revert Unauthorized(sender);
    }

    (uint256 minDeposit, uint256 maxDeposit) = IFeeManager(_kernel.
        feeManager).getDepositBounds(address(this));

    uint256 firstDepositAmount = IFeeManager(_kernel.feeManager).
        getFirstDeposit(address(this));

    if (assets == 0 || assets < minDeposit || assets > maxDeposit) {
        revert AmountOutOfBounds(assets, minDeposit, maxDeposit);
    }
    if (!firstDeposits[sender] && assets < firstDepositAmount) {
        revert AmountLessThanMin(assets, firstDepositAmount);
    }
}
```

File: [contracts/liquid/abstract/AsyncStrategy.sol#L167-L184](#)

```

/**
 * @dev Internal function to process a deposit queue
 */
function _deposit(address _sender, address _receiver, uint256 _assets)
    internal virtual {
    uint256 currentId = depositsCounter.current();

    deposits[currentId] = Deposit({
        sender: _sender,
        receiver: _receiver,
        assets: _assets,
        pending: _assets,
        status: ICoreStrategy.Status.Pending,
        timestamp: block.timestamp
    });

    usc.safeTransferFrom(_sender, address(this), _assets);

    depositsCounter.increment();

    emit DepositRequested(currentId, _sender, _receiver, _assets);
}

```

File: [contracts/liquid/abstract/AsyncStrategy.sol#L54-L96](#)

```

/**
 * @inheritdoc IAsyncStrategy
 */
function processDeposits(uint256[] calldata _ids) external virtual {
    RoleChecker.hasRole(kernel.roleManager, operatorRole());

    oracle.checkIfPriceIsStale();

    for (uint256 i = 0; i < _ids.length; i++) {
        uint256 id = _ids[i];

        Deposit storage d = deposits[id];

        address sender = d.sender;

        if (sender == address(0)) {
            revert InvalidRequest();
        }
        if (d.status != ICoreStrategy.Status.Pending) {
            revert InvalidState();
        }
    }
}

```

```

uint256 pending = d.pending;

uint256 fee = getFee(ICoreStrategy.ActionType.DEPOSIT, pending,
    sender);

uint256 payout = pending - fee;

uint256 shares = _convertToShares(payout);

if (shares == 0) {
    revert InvalidRequest();
}

d.pending = 0;
d.status = ICoreStrategy.Status.Processed;

if (fee > 0) usc.safeTransfer(feeTreasury, fee);

_processDeposit(d.receiver, payout, shares);

if (!firstDeposits[d.sender]) firstDeposits[d.sender] = true;

emit ProcessDeposit(id, sender, d.receiver, pending, shares, fee,
    treasury, feeTreasury);
}
}

```

File: [contracts/liquid/abstract/CoreStrategy.sol#L290-L296](#)

```

/**
 * @inheritdoc ICoreStrategy
 */
function redeem(uint256 shares, address receiver) external virtual {
    address sender = msg.sender;

    _validateWithdrawalRequest(sender, receiver, shares);
    // process withdrawal request creation
    _withdraw(sender, receiver, shares);
}

```

File: [contracts/liquid/abstract/CoreStrategy.sol#L354-L403](#)

```

/**
 * @inheritdoc ICoreStrategy
 */
function processWithdrawals(uint256[] calldata ids) external virtual {
    RoleChecker.hasRole(kernel.roleManager, operatorRole());

    oracle.checkIfPriceIsStale();

    uint256 timeframe = getTimeframe(block.timestamp);

    IController _controller = IController(kernel.controller);

    for (uint256 i = 0; i < ids.length; i++) {
        uint256 id = ids[i];

        Withdrawal memory withdrawal = withdrawals[id];

        if (withdrawal.sender == address(0)) {
            revert InvalidRequest();
        }
        if (withdrawal.status != ICoreStrategy.Status.Pending) {
            revert InvalidState();
        }
        if (_controller.isBlacklisted(withdrawal.sender) || _controller.isBlacklisted(withdrawal.receiver)) {
            revert AddressBlacklisted();
        }

        uint256 shares = withdrawal.shares;

        // @note: assets amount is calculated based on the current price
        uint256 assets = _convertToAssets(shares);

        if (assets == 0) {
            revert InvalidRequest();
        }
    }
}

```

```

    // check if available assets are sufficient
    if (assets > liquidAssets[timeframe]) {
        revert InsufficientLiquidAssets();
    }

    uint256 fee = getFee(ActionType.REDEEM, assets, withdrawal.sender);

    withdrawals[id].status = ICoreStrategy.Status.Processed;

    // deduct `assets` before fees
    liquidAssets[timeframe] -= assets;

    _processWithdrawal(withdrawal.receiver, address(this), assets,
        shares, fee);

    emit ProcessWithdrawal(id, withdrawal.sender, withdrawal.receiver,
        assets, shares, fee, treasury, feeTreasury);
}
}

```

Impact

- Users commit assets based on current previews, but are settled with different fee rates and prices.
- Fees (globally or per-account) can be changed a lot after the user deposits/locks funds in the queue.

Recommendation

Add slippage and fee guards to the async flow:

For deposits:

- At `deposit()` time:
- Let the user pass `minShares` (and optionally `maxFeeBps` and `deadline`).
- Store those values in the `deposits` mapping alongside `assets`.
- At `processDeposits()` time:
- Recompute fee and shares as of today, but:
- `require(shares >= d.minShares, "slippage")`
- If stored, `require(effectiveFeeBps <= d.maxFeeBps, "fee changed")`
- `require(block.timestamp <= d.deadline, "expired")`

For withdrawals:

- At `redeem()` time: - Let the user pass `minAssets` (and optionally `maxFeeBps` and `deadline`).
- Store them in the `withdrawals` mapping.
- At `processWithdrawals()` time:

- After converting shares to assets and applying fees:
- `require(assetsAfterFee >= w.minAssets, "slippage")`
- Enforce `maxFeeBps` and `deadline` similarly.

Optionally, for stronger guarantees, cache the fee rate at request time and reuse it in processing, so fee changes only affect future requests, not already queued ones.

Team Response

Acknowledged.

[M-03] Users Cannot Withdraw Full Balance When It Falls Below

`minWithdrawRetry`

Severity

Medium Risk

Description

The `_validateWithdrawalRequest()` function enforces `minWithdraw` without allowing users to withdraw their full remaining balance when it falls below this threshold. This creates two scenarios where users lose access to their funds:

Scenario 1: Parameter Change Lockup When the `FeeManager` operator increases `minWithdraw`, users holding shares between the old and new minimum cannot withdraw. For example, if `minWithdraw` changes from 10 to 50 shares, a user with 10 shares is permanently locked out.

Scenario 2: Dust Amount Lockup After partial withdrawals, users may have remaining balances below `minWithdraw`. For example:

- User holds 95 shares with `minWithdraw` = 10
- User withdraws 90 shares (valid)
- User now has 5 shares remaining
- User cannot withdraw the final 5 shares

Both scenarios result from the same validation logic that does not exempt full balance withdrawals from the minimum check.

Location of Affected Code

File: [contracts/liquid/abstract/CoreStrategy.sol#L662-L678](#)

```

function _validateWithdrawalRequest(address _sender, address _receiver,
uint256 _shares) internal view {
    Kernel storage k = kernel;

    IController _controller = IController(k.controller);

    _controller.checkWithdrawal(address(this));

    if (!_controller.hasAccess(_sender, address(this)) || !_controller.
        hasAccess(_receiver, address(this))) {
        revert Unauthorized(_sender);
    }

    (uint256 minWithdraw, uint256 maxWithdraw) = IFeeManager(k.feeManager
        ).getWithdrawBounds(address(this));

    if (_shares == 0 || _shares < minWithdraw || _shares > maxWithdraw) {
        revert AmountOutOfBounds(_shares, minWithdraw, maxWithdraw);
    }
}

```

Impact

Users lose permanent access to their funds in two ways:

1. Administrative parameter changes can trap existing positions
2. Normal partial withdrawals create unrecoverable dust amounts

Both `redeem()` and `redeemInstant()` are affected. While individual locked amounts may be small in scenario 2, scenario 1 can lock significant balances. The issue affects any user making withdrawals.

Recommendation

Add a check to allow users to withdraw their entire balance regardless of the minimum threshold:

```

function _validateWithdrawalRequest(address _sender, address _receiver,
uint256 _shares) internal view {
    Kernel storage k = kernel;

    IController _controller = IController(k.controller);

    _controller.checkWithdrawal(address(this));

    if (!_controller.hasAccess(_sender, address(this)) || !_controller.
        hasAccess(_receiver, address(this))) {
        revert Unauthorized(_sender);
    }

    (uint256 minWithdraw, uint256 maxWithdraw) = IFeeManager(k.feeManager
        ).getWithdrawBounds(address(this));

    uint256 userBalance = balanceOf(_sender);
    bool isFullWithdrawal = (_shares == userBalance);

    if (_shares == 0 || (!isFullWithdrawal && _shares < minWithdraw) ||
        _shares > maxWithdraw) {
        revert AmountOutOfBounds(_shares, minWithdraw, maxWithdraw);
    }
}

```

This allows users to always exit their full position while maintaining minimum withdrawal requirements for partial withdrawals.

Team Response

Fixed.

[L-01] `FeeManager` : `maxDeposit` Can Be Misconfigured < `firstDepositAmount` (DoS on First Deposit)

Severity

Low Risk

Description

`FeeManager.setConfig()` allows setting `firstDepositAmount` larger than `maxDeposit`.

When that happens, any first-time depositor is effectively blocked:

- `assets` must be `<= maxDeposit` to pass the max deposit check
- but also `>= firstDepositAmount` to pass the first-deposit check

If `firstDepositAmount > maxDeposit`, there is no valid `assets` value that satisfies both, so new users can never make their first deposit until the config is fixed.

Location of Affected Code

File: [contracts/liquid/controls/FeeManager.sol#L61-L85](#)

```
/**
 * @inheritdoc IFeeManager
 */
function setConfig(address _address, FeeConfig memory _configurations)
    external {
    RoleChecker.hasRole(roleManager, FEE_MANAGER_OPERATOR_ROLE);

    if (_address == address(0)) {
        revert ZeroAddress();
    }
    if (_configurations.minDeposit >= _configurations.maxDeposit) {
        revert InvalidMinMaxValues();
    }
    if (_configurations.minWithdraw >= _configurations.maxWithdraw) {
        revert InvalidMinMaxValues();
    }
    if (
        _configurations.depositFeeRate > DEFAULT_MAX_FEE_BPS ||
        _configurations.redeemFeeRate > DEFAULT_MAX_FEE_BPS ||
        _configurations.instantRedeemFeeRate > DEFAULT_MAX_FEE_BPS ||
        _configurations.cancelDepositFeeRate > DEFAULT_MAX_FEE_BPS
    ) {
        revert InvalidFeeRate();
    }

    configurations[_address] = _configurations;

    emit ConfigSet(_address);
}
```

Impact

Misconfiguration can fully DoS first deposits for a strategy:

- Existing users can continue to operate (if they don't hit the first-deposit check).
- New users are blocked from entering the system until an admin notices and corrects the config.

Recommendation

In `FeeManager.sol##setConfig()`, add an explicit invariant:

```
require(
    config.maxDeposit >= config.firstDepositAmount,
    "FeeManager: maxDeposit < firstDepositAmount"
);
```

Team Response

Fixed.

[L-02] `LSFactory` Emits Outdated Implementation Address After Beacon Upgrade

Severity

Low Risk

Description

`LSFactory` stores the implementation address only once during initialization.

Afterwards, the actual logic used by all strategies comes from the beacon (`UpgradeableBeacon`).

If the admin upgrades the beacon to a new implementation, newly deployed strategies will run the upgraded logic, but `LSFactory` will continue to emit events referencing the old implementation stored in its immutable state.

So every time `deploy()` is called post-upgrade:

- `BeaconProxy` delegates to `newImpl`
- `LSDeployed` event still reports `oldImpl`

This creates a permanently misleading on-chain log trail.

Location of Affected Code

File: `contracts/liquid/LSFactory.sol`

```
// LSFactory.sol
implementation = _implementation;

// later in deploy()
emit LSDeployed(address(strategy), implementation);
```

Impact

Off-chain indexers, monitoring systems, auditors and users relying on `LSDeployed` logs will be misled, believing strategies are deployed using the old implementation when in reality they use the upgraded beacon implementation.

Recommendation

Emit the current beacon implementation, not the stale factory-stored value:

```
emit LSDeployed(address(strategy), UpgradeableBeacon(beacon).
    implementation());
```

Or remove the implementation argument entirely and make indexers query the beacon directly.

Team Response

Fixed.

[L-03] Missing Storage Gap in AsyncStrategy

Severity

Low Risk

Description

The `AsyncStrategy` abstract contract is missing a storage gap. This contract contains state variables and is inherited by `PreIPOStrategy` and `RWAStrategy`. Without a storage gap, adding new state variables to `AsyncStrategy` during an upgrade will corrupt the storage layout of derived contracts, overwriting state variables like role identifiers.

Location of Affected Code

File: `contracts/liquid/abstract/AsyncStrategy.sol`

```
abstract contract AsyncStrategy is CoreStrategy, IAsyncStrategy {
    using MathUpgradeable for uint256;
    using SafeERC20Upgradeable for IERC20Upgradeable;
    using Counters for Counters.Counter;

    /*//////////////////////
                        STATE
    ////////////////////////*/

    /**
     * @notice Counter for deposit request IDs
     */
    Counters.Counter internal depositsCounter;

    /**
     * @notice Mapping from deposit ID to deposit data
     */
    mapping(uint256 => Deposit) public deposits;

    // MISSING: uint256[50] private __gap;
}
```

Impact

If `AsyncStrategy` is upgraded with new state variables, those variables will occupy storage slots currently used by derived contracts like `RWAStrategy`.

Storage Layout Example:

Current layout:

- CoreStrategy variables + gap: slots 0 to X
- AsyncStrategy.depositsCounter: slot X+1
- AsyncStrategy.deposits: slot X+2

- RWAStrategy._operatorRole: slot X+3
- RWAStrategy._assetManagerRole: slot X+4

After adding a variable to AsyncStrategy without a gap:

- AsyncStrategy.newVariable: slot X+3 (overwrites _operatorRole)

This corrupts the `_operatorRole` variable. The operator role hash will be overwritten with the new variable's data.

Recommendation

Add a storage gap to `AsyncStrategy`:

```
abstract contract AsyncStrategy is CoreStrategy, IAsyncStrategy {
    using MathUpgradeable for uint256;
    using SafeERC20Upgradeable for IERC20Upgradeable;
    using Counters for Counters.Counter;

    /*////////////////////STATE////////////////////////////////////*/
    /*
     * @notice Counter for deposit request IDs
     */
    Counters.Counter internal depositsCounter;

    /**
     * @notice Mapping from deposit ID to deposit data
     */
    mapping(uint256 => Deposit) public deposits;

    /**
     * @notice Storage gap for future upgrades
     */
    uint256[50] private __gap;
}
```

Team Response

Fixed.

[L-04] Improper Naming of Withdraw Bounds (Deposit Bounds Are Checked in Assets)

Severity

Low Risk

Description

The deposit min/max bounds are set in assets (USC)

Deposits from `FeeManager` are defined in terms of USC amounts and checked in USC amounts during the deposit flow. From the name of withdrawal bounds in `FeeManager`, it is understood that they are also defined in terms of USC amounts, but the `CoreStrategy.sol##_validateWithdrawalRequest()` compares them directly against `shares`. That's fine for the current strategy design.

The problem is future-proofing: - `FeeManager` config is explicitly defined in assets (USC), not shares. Anyone adding a new strategy will naturally assume `minWithdraw/maxWithdraw` are limits in assets, because that's how the config is documented. - If a new strategy uses a different pricing model, or a non-1:1 bootstrap price, the current behavior silently misapplies bounds. A config thinking "min withdrawal = 50 USC" becomes "min withdrawal = 50 shares", which might represent 500, 5,000, or 0.5 USC depending on price. - The check is inconsistent with deposit bounds (assets). Mixed semantics inside the same fee/config system is a maintainability trap.

Location of Affected Code

File: `contracts/liquid/abstract/CoreStrategy.sol#L662-L678`

```
function _validateWithdrawalRequest(address _sender, address _receiver,
    uint256 _shares) internal view {
    Kernel storage k = kernel;

    IController _controller = IController(k.controller);

    _controller.checkWithdrawal(address(this));

    if (!_controller.hasAccess(_sender, address(this)) || !_controller.
        hasAccess(_receiver, address(this))) {
        revert Unauthorized(_sender);
    }

    (uint256 minWithdraw, uint256 maxWithdraw) = IFeeManager(k.feeManager)
        .getWithdrawBounds(address(this));

    if (_shares == 0 || _shares < minWithdraw || _shares > maxWithdraw) {
        revert AmountOutOfBounds(_shares, minWithdraw, maxWithdraw);
    }
}
```

Recommendation

Document that withdrawal bounds are always in shares, not assets and rename (`minWithdrawShares`) so that the adding of new strategies will be configured correctly.

Team Response

Fixed.

[I-01] Incomplete Events Across Strategy and Factory Modules

Severity

Informational Risk

Description

Across multiple contracts [`CoreStrategy` , `PreIPOStrategy` , `RWAStrategy` , `Controller` , `FeeManager` , `OracleFactory`], configuration-changing functions emit events that only include new values, while omitting the previous values that were overwritten.

These parameters directly influence core system behaviour (oracle selection, treasury routing, fee routing, access control, liquidity accounting, pause logic, kernel configuration).

In addition, `LiquidAssetsUpdated` emits only the new liquidity value, even though liquid assets are tracked per timeframe. Not emitting the timestamp leaves external systems blind to which epoch the liquidity applies to.

This is a strong auditability and traceability gap across the system.

Location of Affected Code

File: `contracts/liquid/abstract/CoreStrategy.sol`

```
emit SetOracle(_oracle);
emit SetTreasury(_treasury);
emit SetFeeTreasury(_feeTreasury);
emit KernelSet(_controller, _feeManager, _roleManager);
emit LiquidAssetsUpdated(_liquidAssets); // missing timestamp
```

File: `contracts/liquid/controls/Controller.sol`

```
emit RoleManagerSet(_roleManager);
emit WhitelistSet(_whitelist);
emit SanctionsListSet(_sanctionsList);
emit BlacklistSet(_blacklist);
emit AccessLevelSet(_address, _requiredLevel);
emit InstantRedemptionSet(_address, _value);
emit GlobalDepositPauseWindowSet(_start, _end);
emit GlobalWithdrawalPauseWindowSet(_start, _end);
emit AddressDepositPauseWindowSet(_address, _start, _end);
emit AddressWithdrawalPauseWindowSet(_address, _start, _end);
```

File: `contracts/liquid/controls/FeeManager.sol`

```
function setConfig(address _address, FeeConfig memory _configurations)
    external {
    // code
    emit ConfigSet(_address);
}

function setAccountFeeConfig(address _address, address _account,
    AccountFeeConfig memory _config) external {
    // code
    emit AccountFeeConfigSet(_address, _account);
}

function setRoleManager(address _roleManager) external {
    // code
    emit RoleManagerSet(_roleManager);
}
```

File: [contracts/liquid/oracle/OracleFactory.sol](#)

```
function setRoleManager(address _roleManager) external {
    // code
    emit RoleManagerSet(_roleManager);
}

function deploy(string memory name, uint256 maxStaleness, uint256
    maxPriceDeviation) external {
    // code
    emit OraclePairDeployed(currentId, aggregatorAddress,
        priceOracleAddress, name, maxStaleness);
}
```

General pattern: All events expose only new values and never include the overwritten ones.

Recommendation

1. Emit old + new values for all configuration setters

For every setter event, include both:

- the value before the change
- the updated value

Example fix:

```
address oldOracle = address(oracle);
oracle = ILSPriceOracle(_oracle);
emit SetOracle(oldOracle, _oracle);
```

Apply the same pattern to:

- `setTreasury()`
- `setFeeTreasury()`

- `setOracle()`
- `setKernel()`
- `setRoleManager()` (Controller, FeeManager, OracleFactory)
- `setWhitelist()`, `setBlacklist()`, `setSanctionsList()`
- `setAccessLevel()`
- `setInstantRedemption()`
- `ConfigSet`, `AccountFeeConfigSet`
- Pause window setters

2. Add a timestamp to `LiquidAssetsUpdated`:

```
emit LiquidAssetsUpdated(_liquidAssets, block.timestamp);
```

Team Response

Fixed.

[I-02] Deprecated OpenZeppelin Counters Library Usage

Severity

Informational Risk

Description

The `CoreStrategy.sol` and `AsyncStrategy.sol` contracts use OpenZeppelin's Counters library for tracking deposit and withdrawal IDs. This library was deprecated in OpenZeppelin Contracts v4.9.0 and subsequently removed in v5.0.0.

The OpenZeppelin team deprecated this library because it provides no real benefit over using a simple uint256 variable with direct increment operations. The library adds unnecessary abstraction, slightly increases gas costs, and creates a false sense of security without providing meaningful overflow protection (which is already built into Solidity 0.8+). Using deprecated libraries introduces technical debt and potential compatibility issues when upgrading dependencies in the future.

Location of Affected Code

File: `contracts/liquid/abstract/CoreStrategy.sol`

```
import {Counters} from "@openzeppelin/contracts/utils/Counters.sol";

using Counters for Counters.Counter;

Counters.Counter internal withdrawalsCounter;
```

File: `contracts/liquid/abstract/AsyncStrategy.sol`

```
import {Counters} from "@openzeppelin/contracts/utils/Counters.sol";

using Counters for Counters.Counter;

Counters.Counter internal depositsCounter;
```

Impact

Increased Gas Costs: The library abstraction adds unnecessary overhead compared to native uint256 operations

Recommendation

Remove the Counters library import and replace all Counter type declarations with simple uint256 variables. Update the usage pattern by replacing `.current()` calls with direct variable reads and `.increment()` calls with standard increment operations. Consider wrapping increments in an unchecked block for gas optimization since overflow is practically impossible for a counter that would need 2^{256} increments.

Team Response

Fixed.

[I-03] State Variable `lastUpdateAt` Set But Never Exposed

Severity

Informational Risk

Description

The `AsyncStrategy` abstract contract is missing a storage gap. This contract contains state variables and is inherited by `PreIPOStrategy` and `RWAStrategy`. Without a storage gap, adding new state variables to `AsyncStrategy` during an upgrade will corrupt the storage layout of derived contracts, overwriting state variables like role identifiers.

Location of Affected Code

File: `contracts/liquid/abstract/AsyncStrategy.sol`

```

abstract contract AsyncStrategy is CoreStrategy, IAsyncStrategy {
    using MathUpgradeable for uint256;
    using SafeERC20Upgradeable for IERC20Upgradeable;
    using Counters for Counters.Counter;

    /*//////////////////////////////////////
        STATE
    //////////////////////////////////////*/

    /**
     * @notice Counter for deposit request IDs
     */
    Counters.Counter internal depositsCounter;

    /**
     * @notice Mapping from deposit ID to deposit data
     */
    mapping(uint256 => Deposit) public deposits;

    // MISSING: uint256[50] private __gap;
}

```

Impact

If `AsyncStrategy` is upgraded with new state variables, those variables will occupy storage slots currently used by derived contracts like `RWAStrategy`.

Storage Layout Example:

Current layout:

- CoreStrategy variables + gap: slots 0 to X
- AsyncStrategy.depositsCounter: slot X+1
- AsyncStrategy.deposits: slot X+2
- RWAStrategy._operatorRole: slot X+3
- RWAStrategy._assetManagerRole: slot X+4

After adding a variable to AsyncStrategy without a gap:

- AsyncStrategy.newVariable: slot X+3 (overwrites _operatorRole)

This corrupts the `_operatorRole` variable. The operator role hash will be overwritten with the new variable's data.

Recommendation

Add a storage gap to `AsyncStrategy`:

```

abstract contract AsyncStrategy is CoreStrategy, IAsyncStrategy {
    using MathUpgradeable for uint256;
    using SafeERC20Upgradeable for IERC20Upgradeable;
    using Counters for Counters.Counter;

    /*//////////////////////
                        STATE
    ////////////////////////*/

    /**
     * @notice Counter for deposit request IDs
     */
    Counters.Counter internal depositsCounter;

    /**
     * @notice Mapping from deposit ID to deposit data
     */
    mapping(uint256 => Deposit) public deposits;

    /**
     * @notice Storage gap for future upgrades
     */
    uint256[50] private __gap;
}

```

Team Response

Fixed.

[I-04] Missing Shares Check in `CoreStrategy::deposit()`

Severity

Informational Risk

Description

The `deposit()` function does not verify that the deposited assets will result in non-zero shares before accepting the deposit.

The check exists in `processDeposits()` but not at the deposit request stage:

```

// In processDeposits() - check exists here
uint256 shares = _convertToShares(payout);
if (shares == 0) {
    revert InvalidRequest();
}

```

Location of Affected Code

File: `contracts/liquid/abstract/CoreStrategy.sol`

```
function deposit(uint256 assets, address receiver) public virtual {
    // ... validation checks ...

    if (assets == 0 || assets < minDeposit || assets > maxDeposit) {
        revert AmountOutOfBounds(assets, minDeposit, maxDeposit);
    }
    // Missing: check if _convertToShares(assets) == 0
}
```

Recommendation

Add an early validation check in the `deposit()` function:

```
function deposit(uint256 assets, address receiver) public virtual {
    // ... existing validation ...

    if (_convertToShares(assets) == 0) {
        revert InvalidRequest();
    }
}
```

Team Response

Fixed.

[I-05] `LSPPriceOracle.sol:getQuote()` Function Does Not Enforce Staleness

Severity

Informational Risk

Description

The `LSPPriceOracle.sol#getQuote()` function reads `latestRoundData()` but never checks whether the price is stale. Staleness is only enforced via a separate `checkIfPriceIsStale()` call and some core paths call it, but any view logic or off-chain consumer directly using `getQuote` can silently operate on outdated prices if nobody calls the staleness check first.

Location of Affected Code

File: [contracts/liquid/oracle/LSPPriceOracle.sol#L63-L75](#)

```
function getQuote(uint256 amount) external view returns (uint256) {
    if (amount == 0) {
        revert InvalidAmount();
    }

    (, int256 _answer,,, ) = aggregator.latestRoundData();

    if (_answer <= 0) {
        revert InvalidPrice();
    }

    return Math.mulDiv(uint256(_answer), amount, 10 ** aggregator.
        decimals());
}
```

File: [contracts/liquid/oracle/LSPriceOracle.sol#L80-L84](#)

```
function checkIfPriceIsStale() external view {
    (, , , uint256 updatedAt, ) = aggregator.latestRoundData();

    if ((block.timestamp - updatedAt) > maxStaleness) revert StalePrice()
    ;
}
```

Impact

If the underlying Chainlink feed stops updating, `getQuote()` will still return a price.

Recommendation

Two options:

- Strict: move the staleness check into `getQuote` itself:

```
function getQuote(uint256 amount) external view returns (uint256) {
    if (amount == 0) revert InvalidAmount();

    (, int256 _answer, , uint256 updatedAt, ) = aggregator.
        latestRoundData();
    if (_answer <= 0) revert InvalidPrice();
    if (block.timestamp - updatedAt > maxStaleness) revert StalePrice();

    return Math.mulDiv(uint256(_answer), amount, 10 ** aggregator.
        decimals());
}
```

- Or: keep behavior as-is but clearly document that consumers must call `checkIfPriceIsStale()` [or tolerate staleness] when using `getQuote` for anything safety-critical.

Team Response

Fixed.

[I-06] Misconfiguration Risk: `isConfigured()` / Account `isConfigured()` Must Be Manually Set in `FeeManager` Contract

Severity

Informational Risk

Description

Both global and per-account fee configs rely on an `isConfigured()` flag that must be set manually by the caller. If the operator forgets to set `isConfigured = true` when calling `setConfig` or `setAccountFeeConfig()`, the stored values are either treated as “not configured” (global) or silently ignored (per-account). That’s an easy way to DoS a strategy or misapply fee settings for specific accounts.

Location of Affected Code

File: `contracts/liquid/controls/FeeManager.sol`

```

mapping(address => FeeConfig) public configurations;
mapping(address => mapping(address => AccountFeeConfig)) public
    accountFeeConfigs;

function setConfig(address _address, FeeConfig memory _configurations)
    external {
    // code
    configurations[_address] = _configurations;
    emit ConfigSet(_address);
}

function setAccountFeeConfig(address _address, address _account,
    AccountFeeConfig memory _config) external {
    // code
    accountFeeConfigs[_address][_account] = _config;
    emit AccountFeeConfigSet(_address, _account);
}

function _checkIsConfigured(address _address) private view {
    if (!configurations[_address].isConfigured) revert NotConfigured();
}

function getFeeRate(ICoreStrategy.ActionType actionType, address _address
    , address account) external view returns (uint16) {
    _checkIsConfigured(_address);

    AccountFeeConfig storage ac = accountFeeConfigs[_address][account];

    if (actionType == ICoreStrategy.ActionType.DEPOSIT) {
        return ac.isConfigured ? ac.depositFeeRate : configurations[
            _address].depositFeeRate;
    }
    // code
}

```

Recommendation

Make `isConfigured` the FeeManager's responsibility:

- Force-set it to `true` in setters:

```

function setConfig(address _address, FeeConfig memory _configurations)
    external {
    // code
    _configurations.isConfigured = true;
    configurations[_address] = _configurations;
    emit ConfigSet(_address);
}

```

```
function setAccountFeeConfig(address _address, address _account,
    AccountFeeConfig memory _config) external {
    // code
    _config.isConfigured = true;
    accountFeeConfigs[_address][_account] = _config;
    emit AccountFeeConfigSet(_address, _account);
}
```

- If you need to “unset”, create explicit `clearConfig()` / `clearAccountFeeConfig()` functions instead of overloading `set*`.

Team Response

Acknowledged.

our shielding . Your smart contracts, our shielding . Your smart c



shieldify



Thank you!

